

Activation Sparsification in Transformers: Pressure, Thresholding, and Site Placement

Working title — introduction and related work — 5 September 2026

1 Introduction

Activation sparsity offers a route to reducing the cost of Transformer inference. For a linear transformation $y = xW$, a zero coordinate of x nullifies the corresponding products, which a specialized kernel can skip. Because activations depend on the input, the available savings can vary across tokens, layers, and model components. Training-free methods such as TEAL show that pretrained language models can tolerate setting small-magnitude activations to zero and that this sparsity can yield measured decoding speedups (Liu et al., 2024). This establishes a useful starting point: a learned representation already contains some capacity for sparse execution. Intervening during pre-training raises a further question: can the learning process shape that capacity to achieve greater sparsity at the same model quality?

Several approaches establish that adaptation to sparsity is possible. ProSparse combines ReLU activations, progressive ℓ_1 regularization, and threshold shifting during continued training (Song et al., 2024). Cetin et al. (2026) induce highly sparse FFN activations during training and develop kernels that exploit them. Q-Sparse extends training-time sparsification to linear projections in both attention and FFNs through top- k selection (Wang et al., 2024). These results motivate studying the ingredients of sparsification separately. A successful combination establishes an operating point, but explaining that point requires knowing which ingredients contribute and how their effects depend on one another.

We study activation sparsification as three separable interventions: *pressure*, which encourages selected activations toward zero; *thresholding*, which determines which values become exactly zero; and *site placement*, which determines where each intervention acts. Our central question is: *how does the effect of activation pressure on the quality–sparsity trade-off depend on the gate nonlinearity and the sites that receive pressure?* Separability here is experimental: pressure targets can be changed while gates remain fixed, and gates can be changed without adding pressure. Their effects need not be additive. In particular, extending the gate set and extending the pressure objective are distinct changes, even when a practical recipe applies both at the same sites.

A candidate explanation follows from the different roles of these interventions. Pressure reshapes activation magnitudes; the gate converts some of that change into exact zeros; and the downstream operation determines the computational value of those zeros. Moving an activation toward zero need not remove a product until it crosses a gate’s threshold. Moreover, a one-sided gate removes negative values, whereas a symmetric magnitude gate preserves sufficiently large values of either sign. The same pressure can therefore interact with different gates in different ways. Its quality cost also depends on how the auxiliary update interacts with task learning. This account motivates tracing an intervention from optimization, through sitewise activation changes, to operation-level sparsity, and testing whether that account predicts behavior outside the setting in which it was developed.

To connect local behavior to the complete forward pass, we use R_{model} , the fraction of declared model-wide matrix-multiplication products with an exactly zero activation operand. The count includes the Transformer projections and causally valid attention products, with the dense language-model head retained in the denominator. Throughout, *model-wide sparsity* refers to this product-weighted quantity; local activation zero fractions are reported separately. We also define $R_{\text{model}}^{\max}$, the analytic all-zero reach ceiling of a selected site topology for a specified architecture and sequence length. It describes the work reachable through those sites, independently of learned activation values. This separates the opportunity offered by a topology from the sparsity realized by a trained model. Neither quantity is a wall-clock measurement.

We organize the study around a *sparsification intervention ladder* (Figure 1), progressing from a stock GeLU model through FFN-local ReLU and pressure to gates at branch inputs, the attention context, and query, key, and value operands. The ladder provides a map of interventions; attribution comes from matched contrasts within it. Our experimental setting uses randomly initialized Pythia-family architectures at 14M, 70M, and 410M parameters on MiniPile (Biderman et al., 2023; Kaddour, 2023), with initialization, data order, and training budget matched within each comparison. The detailed small-model contrasts and selected larger-model extensions serve different roles: the former examine individual interventions, while the latter probe their transfer under a fixed data budget. Model size also changes the operation weights in R_{model} , so a larger sparsity fraction alone cannot establish a stronger learned response.

The paper is organized around three empirical questions. First, when does adding pressure improve the quality–sparsity trade-off, and how does that effect change with gate and pressure placement? Second, can activation and optimization diagnostics explain the resulting operation-level changes? Third, which parts of that explanation transfer across model sizes, and under what execution conditions can the resulting zeros be exploited? Together, these questions make the interaction, its explanation, and its limits the objects of study.

2 Related Work

Post-hoc activation sparsification. CATS applies magnitude thresholding within gated FFNs and studies both training-free use and subsequent fine-tuning (Lee et al., 2024). TEAL applies magnitude thresholding across attention and FFN projections, studies input versus output sparsity, and uses block-wise greedy allocation to accommodate heterogeneous sensitivity (Liu et al., 2024). Thus, both the placement of sparsity and its allocation across projections have precedents. Our question concerns how those choices interact with activation pressure during pretraining. Post-hoc thresholding provides a reference for the trade-off available without changing the learned weights. Uniform TEAL-style thresholding is a restricted reference and does not reproduce TEAL’s optimized allocation procedure.

Learning sparse representations. ReLU Strikes Back studies restoring sparse activations in language models (Mirzadeh et al., 2023). ProSparse adds progressive ℓ_1 pressure and threshold shifting, including ablations showing that the regularization schedule affects the quality obtained at comparable FFN sparsity (Song et al., 2024). Sparsing Law further studies the dependence of activation sparsity on model size, training data, and regularization strength (Luo et al., 2025). Cetin et al. (2026) combine FFN activation regularization with sparse execution for training and inference. These works establish that sparsity depends on the training procedure as well as the activation function. Their FFN activation fractions, however, measure a different quantity from a product count that includes attention and the dense output head. We build on this literature by separating pressure from the gates and sites through which it affects the complete forward pass.

Sparsity across attention and FFNs. Q-Sparse trains with top- k selection on inputs to attention and FFN linear projections, using a straight-through estimator and squared ReLU; it studies training from scratch, continued training, and fine-tuning, and reports overall sparsity and activated-parameter budgets (Wang et al., 2024). It therefore already establishes training-time sparsification beyond FFNs. Its projection input masks should be distinguished from masks on the query, key, and value operands inside attention. Spark Transformer combines sparse FFNs with top- k attention (You et al., 2025); selecting attention positions is also different from zeroing feature coordinates in those operands. Our focus is the marginal effect of pressure and gate placement under a shared training protocol. The contribution sought is an explanation of that interaction, including its dependence on the operation being sparsified.

Task and regularization gradients. PCGrad projects away conflicting gradient components in multi-task learning (Yu et al., 2020). Bloop treats an auxiliary objective as secondary to the training objective and uses a moving average to improve gradient surgery (Hsieh et al., 2024). These methods motivate examining optimization interference when activation pressure is added. Our orthogonal ℓ_1 variant (OL1) keeps AdamW moments task-only, preconditions the pressure direction, projects its conflicting component relative to the adaptive task direction, and caps the correction with a relative trust budget. This makes the handling of pressure an explicit intervention. It does not guarantee unchanged task-loss convergence or remove the need to specify pressure strength and the trust budget.

From zeros to execution. TEAL’s decoding kernels and the TwELL format and fused kernels of Cetin et al. (2026) demonstrate that activation sparsity can yield wall-clock gains in specified workloads (Liu et al., 2024). Their execution designs make packing, memory access, and fusion part of the sparsity problem. KernelBench evaluates correctness and speed in generated GPU kernels (Ouyang et al., 2025), while recent agentic kernel optimization also studies irregular workloads such as mixture-of-experts and sparse attention (Luo et al., 2026). These results motivate kernel adaptation as a separate test of whether a learned zero pattern is useful on a target device. They do not supply a universal conversion from R_{model} to speedup. For this study, logical sparsity and measured execution performance therefore answer complementary questions.

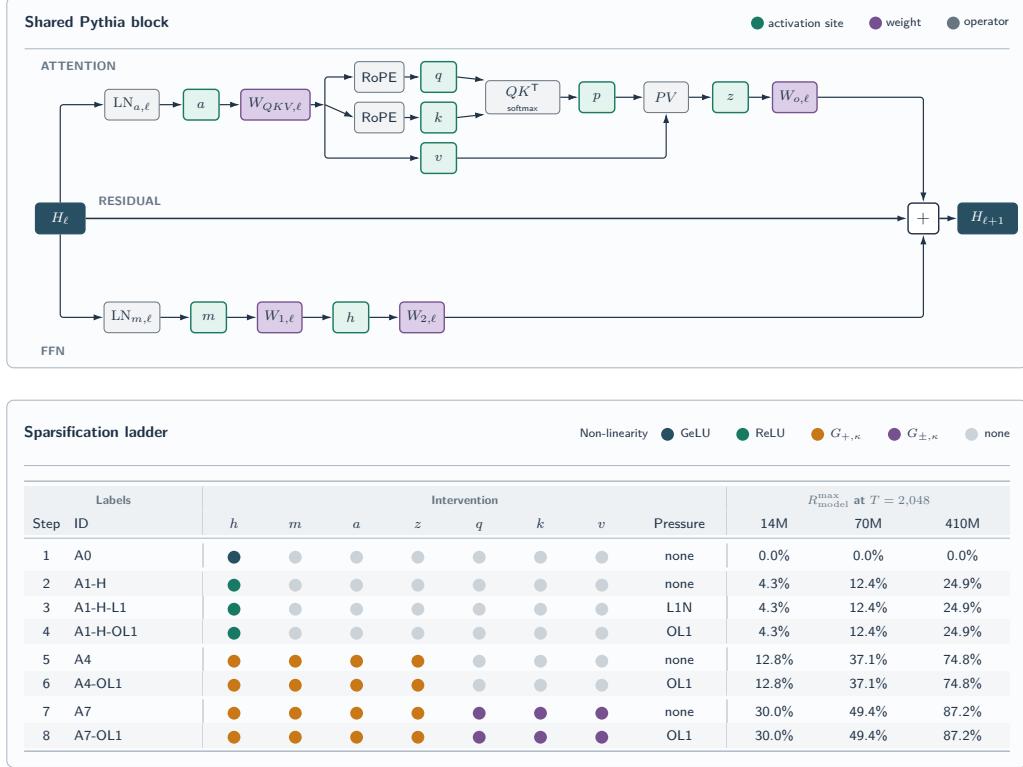


Figure 1: Sparsification intervention ladder and the shared Pythia block. The map distinguishes gate placement, gate family, and pressure method. The ceiling columns are analytic all-zero reach fractions at $T = 2,048$; they are neither observed sparsity nor speedup. The diagram defines candidate conditions, not a claim that every condition was evaluated at every size. In the corrected A4-OL1 and A7-OL1 recipes, pressure covers the four and seven gated sites, respectively; comparing these recipes changes both sets. Query and key gates are after RoPE, and z is the context before the attention output projection.

References

- Stella Biderman, Hailey Schoelkopf, Quentin Anthony, Herbie Bradley, Kyle O’Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, Aviya Skowron, Lintang Sutawika, and Oskar van der Wal. Pythia: A Suite for Analyzing Large Language Models Across Training and Scaling. *Proceedings of the 40th International Conference on Machine Learning*, 2023. URL <https://arxiv.org/abs/2304.01373>.
- Edoardo Cetin, Stefano Peluchetti, Emilio Castillo, Akira Naruse, Mana Murakami, and Llion Jones. Sparser, Faster, Lighter Transformer Language Models. *arXiv preprint arXiv:2603.23198*, 2026. URL <https://arxiv.org/abs/2603.23198>.
- Yu-Guan Hsieh, James Thornton, Eugene Ndiaye, Michal Klein, Marco Cuturi, and Pierre Ablin. Careful with that Scalpel: Improving Gradient Surgery with an EMA. *arXiv preprint arXiv:2402.02998*, 2024. URL <https://arxiv.org/abs/2402.02998>.
- Jean Kaddour. The MiniPile Challenge for Data-Efficient Language Models. *arXiv preprint arXiv:2304.08442*, 2023. URL <https://arxiv.org/abs/2304.08442>.
- Donghyun Lee, Je-Yong Lee, Genghan Zhang, Mo Tiwari, and Azalia Mirhoseini. CATS: Contextually-Aware Thresholding for Sparsity in Large Language Models. *arXiv preprint arXiv:2404.08763*, 2024. URL <https://arxiv.org/abs/2404.08763>.
- James Liu, Pragaash Ponnusamy, Tianle Cai, Han Guo, Yoon Kim, and Ben Athiwaratkun. Training-Free Activation Sparsity in Large Language Models. *arXiv preprint arXiv:2408.14690*, 2024. URL <https://arxiv.org/abs/2408.14690>.
- Mao Luo, Hongbin Li, Feng Lin, Hanling Yi, and Zhe Huang. Agentic Kernel Optimization: Generating State-of-the-Art GPU Kernels Without Hand-Written CUDA. *arXiv preprint arXiv:2608.14560*, 2026. URL <https://arxiv.org/abs/2608.14560>.
- Yuqi Luo, Chenyang Song, Xu Han, Yingfa Chen, Chaojun Xiao, Xiaojun Meng, Liquan Deng, Jiansheng Wei, Zhiyuan Liu, and Maosong Sun. Sparsing Law: Towards Large Language Models with Greater Activation Sparsity. *Proceedings of the 42nd International Conference on Machine Learning*, 2025. URL <https://proceedings.mlr.press/v267/luo25i.html>.
- Iman Mirzadeh, Keivan Alizadeh, Sachin Mehta, Carlo C. Del Mundo, Oncel Tuzel, Golnoosh Samei, Mohammad Rastegari, and Mehrdad Farajtabar. ReLU Strikes Back: Exploiting Activation Sparsity in Large Language Models. *arXiv preprint arXiv:2310.04564*, 2023. URL <https://arxiv.org/abs/2310.04564>.
- Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. KernelBench: Can LLMs Write Efficient GPU Kernels? *arXiv preprint arXiv:2502.10517*, 2025. URL <https://arxiv.org/abs/2502.10517>.
- Chenyang Song, Xu Han, Zhengyan Zhang, Shengding Hu, Xiyu Shi, Kuai Li, Chen Chen, Zhiyuan Liu, Guangli Li, Tao Yang, and Maosong Sun. ProSparse: Introducing and Enhancing Intrinsic Activation Sparsity within Large Language Models. *arXiv preprint arXiv:2402.13516*, 2024. URL <https://arxiv.org/abs/2402.13516>.
- Hongyu Wang, Shuming Ma, Ruiping Wang, and Furu Wei. Q-Sparse: All Large Language Models can be Fully Sparsely-Activated. *arXiv preprint arXiv:2407.10969*, 2024. URL <https://arxiv.org/abs/2407.10969>.
- Chong You, Kan Wu, Zhipeng Jia, Lin Chen, Srinadh Bhojanapalli, Jiaxian Guo, Utku Evci, Jan Wassenberg, Praneeth Netrapalli, Jeremiah J. Willcock, Suvinay Subramanian, Felix Chern, Alek Andrejev, Shreya Pathak, Felix Yu, Prateek Jain, David E. Culler, Henry M. Levy, and Sanjiv Kumar. Spark Transformer: Reactivating Sparsity in FFN and Attention. *arXiv preprint arXiv:2506.06644*, 2025. URL <https://arxiv.org/abs/2506.06644>.
- Tianhe Yu, Saurabh Kumar, Abhishek Gupta, Sergey Levine, Karol Hausman, and Chelsea Finn. Gradient Surgery for Multi-Task Learning. In *Advances in Neural Information Processing Systems*, 2020. URL <https://arxiv.org/abs/2001.06782>.